

Aula 07 - Funções, Escopo e Manipulação de Dados

Descrição: Modularização de código, passagem de parâmetros, retorno de funções e tratamento básico de arrays/estruturas.

Objetivo Geral: Capacitar os estudantes a estruturar código C modularizado, seguro e eficiente para microcontroladores, dominando o gerenciamento de escopo, passagem de parâmetros por referência e tratamento de arrays e estruturas sem alocação dinâmica de memória.

Introdução

Vamos organizar um programa em C em pedaços menores (módulos), onde cada variável "mora" na memória do microcontrolador, como passar dados entre funções sem gastar memória à toa, e por que não podemos usar malloc() em sistemas embarcados.

Ao final desta aula, você vai conseguir dividir um programa em arquivos .h e .c, escolher corretamente onde uma variável deve "morar" (static, volatile, const), passar dados grandes entre funções sem desperdiçar memória, e entender por que a indústria embarcada evita alocação dinâmica.

1. Modularização: por que dividir o código em pedaços?

Imagine escrever um programa inteiro — sensores, comunicação, motores, telas — em um único arquivo main.c com 5 mil linhas. Encontrar um erro nesse arquivo seria como procurar uma agulha no palheiro. A solução é dividir o sistema em módulos: grupos de código que cuidam de uma responsabilidade só (por exemplo, "tudo que envolve o sensor de temperatura").

Pense em um restaurante

O arquivo .h é o cardápio: mostra ao cliente (o resto do programa) o que está disponível, sem revelar a receita.

O arquivo .c é a cozinha: é onde a comida realmente é preparada. O cliente não precisa saber como o prato é feito — só precisa saber o que pode pedir.

1.1 Arquivo de cabeçalho (.h) — a interface pública

O .h contém apenas o que outros arquivos precisam enxergar: os protótipos das funções (o "nome e os ingredientes" de cada função, sem o código de dentro) e os tipos de dados (struct, typedef) usados por quem vai chamar essas funções.

1.2 Arquivo de implementação (.c) — a lógica privada

O .c contém o código real das funções — o "como" — além de variáveis que só aquele módulo precisa usar internamente.

1.3 Include guards: evitando que o cardápio seja lido duas vezes

Se dois arquivos diferentes incluírem o mesmo .h, o compilador pode tentar definir a mesma struct duas vezes e travar com erro de redefinição. As diretivas #ifndef, #define e #endif funcionam como uma etiqueta "já lido": na primeira vez que o arquivo é incluído, o compilador cola a etiqueta; nas próximas vezes, ao ver a etiqueta já colada, ele simplesmente pula o conteúdo.

Exemplos de código (sensor_temp.h e sensor_temp.c):

```
/* sensor_temp.h - Interface Pública (o cardápio) */
#ifndef SENSOR_TEMP_H // "Já coleí essa etiqueta antes?"
#define SENSOR_TEMP_H // Se não, cola a etiqueta agora

#include <stdint.h> // tipos inteiros com tamanho garantido (uint8_t, uint16_t...)
#include <stdbool.h> // tipo booleano (true, false)

typedef struct {
    uint8_t id_sensor;
    float temperatura_celsius;
    bool leitura_valida;
} sensor_temp_t;

void sensor_temp_init(void);
bool sensor_temp_ler(sensor_temp_t * const p_sensor);

#endif /* SENSOR_TEMP_H */
```

```
/* sensor_temp.c - Implementação Privada (a cozinha) */
#include "sensor_temp.h"
#include <stddef.h>

// 'static' = função privada: só este arquivo enxerga
static float converter_adc_para_celsius(uint16_t valor_adc) {
    return ((float)valor_adc * 3.3f / 4095.0f) * 100.0f;
}

void sensor_temp_init(void) {
    /* configura o hardware do sensor */
}

bool sensor_temp_ler(sensor_temp_t * const p_sensor) {
    if (p_sensor == NULL) return false; // nunca confie no ponteiro recebido
    uint16_t valor_bruto_adc = 2048; // leitura simulada
    p_sensor->temperatura_celsius = converter_adc_para_celsius(valor_bruto_adc);
    p_sensor->leitura_valida = true;
    return true;
}
```

2. Onde cada variável "mora": o mapa da memória

Todo microcontrolador tem dois tipos de memória principais: a **Flash** e a **SRAM**.

A **memória Flash** e a **SRAM** desempenham papéis diferentes nos computadores e aparelhos eletrônicos. A memória Flash guarda arquivos e programas de forma permanente (mesmo sem energia), enquanto a SRAM funciona como uma memória de trabalho ultrarrápida e volátil (apaga ao desligar) usada pelo processador.

Entender a diferença entre elas é a chave para entender **static**, **const** e **volatile**.

Flash é um caderno impresso, SRAM é um quadro branco

A Flash guarda dados de forma permanente — como um livro impresso: mesmo desligando o microcontrolador, o conteúdo continua lá. A SRAM é como um quadro branco: rápida de escrever e apagar, mas o conteúdo some quando a energia acaba.

O compilador organiza o programa em 5 "gavetas" de memória:

Gaveta (seção)	O que guarda e onde fica
.text (Flash)	O código compilado, em linguagem de máquina. Só leitura — como as letras já impressas em um livro.
.rodata (Flash)	Constantes (const) e textos fixos. Ficam na Flash para não gastar SRAM à toa.
.data (SRAM, cópia vem da Flash)	Variáveis globais/estáticas já inicializadas com um valor diferente de zero (ex: int limite = 50;). O valor nasce na Flash e é copiado para a SRAM quando a placa liga.
.bss (SRAM)	Variáveis globais/estáticas sem valor inicial (ou iniciadas em zero). O sistema simplesmente zera esse espaço ao ligar.
Stack / Pilha (SRAM)	Variáveis locais e parâmetros de função. Funciona como uma pilha de pratos: cada função empilha seu prato ao ser chamada e o retira ao terminar.

2.1 Os três qualificadores que todo firmware usa



static (dentro de um arquivo) = sala trancada

static em uma variável ou função global esconde esse nome do resto do projeto — só o próprio arquivo .c tem a chave dessa sala. Isso evita que outro módulo mexa em algo que não é da conta dele.



static (dentro de uma função) = post-it que a função deixa colado

Normalmente, uma variável local nasce e morre a cada chamada da função. Com static, a função escreve o valor em um post-it (na SRAM) antes de terminar, e na próxima chamada ela lê esse mesmo post-it em vez de começar do zero. É assim que um contador "lembra" quantas vezes já foi chamado.

volatile = "não confie na memória, vá lá conferir de novo"

Por padrão, o compilador tenta ser esperto: se ele acha que uma variável não mudou, reutiliza o último valor que já tinha guardado, para ganhar velocidade. O problema é que registradores de hardware e interrupções podem mudar um valor sem o compilador perceber. volatile avisa: "não confie na memória cache, vá até o endereço físico e leia de novo, sempre".

const = etiqueta de "isto nunca muda"

Além de impedir que o código altere o valor por engano, const em uma variável global faz o compilador guardá-la direto na Flash (.rodata) em vez de gastar SRAM — útil porque a SRAM costuma ser bem mais escassa que a Flash.

```
#include <stdint.h>

// Registrador do ADC mapeado no endereço físico 0x40012000
// 'volatile' = o hardware pode mudar esse valor a qualquer momento
#define REG_STATUS_ADC (*(volatile uint32_t *) 0x40012000U)

// Tabela que nunca muda -> fica guardada na Flash, não na SRAM
static const float TABELA_CALIBRACAO[3] = {1.02f, 0.98f, 1.00f};

uint32_t registrar_evento(void) {
    // 'static' aqui = a contagem sobrevive entre uma chamada e outra
    static uint32_t total_execucoes = 0U;
    return ++total_execucoes;
}

void aguardar_conversao_hardware(void) {
    // sem 'volatile' o compilador poderia "decorar" esse valor e
    // nunca sair do laço, mesmo o hardware já tendo mudado o registrador
    while ((REG_STATUS_ADC & 0x01U) == 0U) { /* aguardando... */ }
}
```

3. Funções e ponteiros: como entregar dados sem desperdiçar memória

Passar por valor = enviar uma fotocópia. Passar por ponteiro = enviar o endereço da casa

Passar por valor cria uma cópia inteira da variável na pilha. Se a variável for uma struct grande, isso custa memória e tempo. Passar por ponteiro envia só o endereço (4 bytes em um microcontrolador de 32 bits) — como entregar o endereço da casa em vez de reconstruir a casa inteira em outro lugar.

3.1 O perigo do ponteiro pendente (dangling pointer)

Nunca devolva o endereço de uma variável local criada dentro da função. Quando a função termina, a pilha "desmonta" aquele espaço — é como demolir uma casa e continuar usando o endereço dela no GPS: o ponteiro aponta para lixo de memória, e isso pode travar o microcontrolador (HardFault).

3.2 A solução: o chamador entrega o balde vazio

Em vez de a função criar o dado e tentar devolvê-lo, é o código que chama a função (o "chamador") quem reserva o espaço e passa o ponteiro para a função apenas preencher. Esse padrão se chama Caller-Allocated Buffer — como levar seu próprio pote reutilizável para a lanchonete: o pote (a memória) já existe e continua existindo depois; a função só o enche.

```
#include <stdint.h>
#include <stdbool.h>
#include <stddef.h>

typedef struct {
    uint32_t leitura_pressao;
    uint32_t timestamp;
} dados_pressao_t;

// CORRETO: quem chama entrega a struct (o 'balde'),
// a função só preenche os dados dentro dele
bool ler_pressao_seguro(dados_pressao_t * const p_saida) {
    if (p_saida == NULL) return false; // proteção contra ponteiro nulo
    p_saida->leitura_pressao = 1013U;
    p_saida->timestamp = 5000U;
    return true;
}
```

4. Arrays e structs: tamanho e organização importam

4.1 Array decay: o array "esquece" seu próprio tamanho

Quando um array é passado para uma função, ele vira apenas um ponteiro para o primeiro elemento — a informação de "quantos elementos ele tinha" se perde no caminho. Por isso, é preciso sempre enviar o tamanho junto, em um parâmetro separado (size_t tamanho).

4.2 Padding: por que a ordem dos campos de uma struct muda o tamanho dela

Processadores de 32 bits leem a memória em blocos alinhados de 4 bytes. Se os campos de uma struct estiverem em uma ordem "desorganizada" (misturando tamanhos pequenos e grandes), o compilador insere bytes de preenchimento invisíveis (padding) para manter esse alinhamento — e a struct acaba maior do que o necessário.

Organizando a mala de viagem

Imagine guardar objetos grandes e pequenos em uma mala sem planejamento: sobram buracos entre eles. Se você primeiro coloca os objetos grandes e só depois encaixa os pequenos nos espaços que sobraram, a mala fica muito mais compacta. Com structs é a mesma lógica: declarar os campos do maior para o menor evita espaço desperdiçado.

```
// Ocupa 12 bytes na RAM (o compilador insere 6 bytes de padding)
struct Desalinhada {
    uint8_t id;    // 1 byte (+ 3 bytes de padding invisível)
    uint32_t valor; // 4 bytes
    uint8_t status; // 1 byte (+ 3 bytes de padding no final)
};

// Ocupa só 8 bytes: do maior campo para o menor
struct Otimizada {
    uint32_t valor; // 4 bytes
    uint8_t id;    // 1 byte
    uint8_t status; // 1 byte (+ 2 bytes de padding no final)
};

// Função que recebe um array: sempre com ponteiro + tamanho
float calcular_media(const uint16_t * const p_array, const size_t tamanho) {
    if (p_array == NULL || tamanho == 0U) return 0.0f;
    uint32_t soma = 0U;
    for (size_t i = 0U; i < tamanho; ++i) {
        soma += p_array[i];
    }
    return (float)soma / (float)tamanho;
}
```

5. Por que malloc é proibido em sistemas embarcados

Em um computador comum, malloc() e free() criam e apagam variáveis dinamicamente na memória Heap. Em sistemas embarcados críticos (carros, aviões, dispositivos médicos), isso é proibido pelas diretrizes MISRA C (Diretiva 4.12 e Regra 21.3). Três motivos principais:

- **Imprevisibilidade:** o tempo que o malloc leva para achar espaço livre varia a cada chamada — ruim para sistemas que precisam responder em tempo real.
- **Fragmentação:** alocar e liberar blocos de tamanhos diferentes deixa "buracos" na memória, podendo faltar espaço contínuo mesmo havendo memória livre no total.
- **Risco de colisão:** um vazamento de memória pode fazer o Heap crescer até colidir com a Stack, travando o hardware sem aviso.

Estacionamento com vagas numeradas em vez de "fila livre"

malloc() é como deixar os carros estacionarem em qualquer espaço livre que encontrarem: rápido no começo, mas com o tempo sobram vãos pequenos demais para um carro grande — mesmo havendo espaço total de sobra. A alternativa segura é ter vagas numeradas de tamanho fixo: cada carro pega uma vaga inteira, e devolvê-la é instantâneo e previsível. Isso é o pool de memória estática.

```
#define TAMANHO_BLOCO 32U
#define TOTAL_BLOCOS 8U

typedef struct {
    uint8_t memoria[TOTAL_BLOCOS][TAMANHO_BLOCO]; // pré-alocado na RAM
    bool ocupado[TOTAL_BLOCOS];
} pool_memoria_t;

// Reserva um bloco em tempo fixo e previsível: O(1)
void* pool_alocar(pool_memoria_t * const pool) {
    if (pool == NULL) return NULL;
    for (size_t i = 0U; i < TOTAL_BLOCOS; ++i) {
        if (!pool->ocupado[i]) {
            pool->ocupado[i] = true;
            return (void*)&(pool->memoria[i][0]);
        }
    }
    return NULL; // memória esgotada, com segurança, sem travar o sistema
}
```

Resumo rápido

Conceito	Em uma frase
.h vs .c	Cardápio (interface) vs. cozinha (implementação).
Include guard	Etiqueta "já lido" que evita erro de redefinição.
static (arquivo)	Sala trancada: só aquele arquivo acessa.
static (função)	Post-it que sobrevive entre chamadas.
volatile	"Vá conferir de novo, não confie no que já leu".
const	"Isto nunca muda" — geralmente vai para a Flash.

Conceito	Em uma frase
Ponteiro pendente	Endereço de uma casa já demolida.
Caller-allocated buffer	Leve seu próprio pote; a função só o enche.
Array decay	O array esquece seu tamanho ao entrar em uma função.
Padding	Buracos de alinhamento; organize do maior campo para o menor.
malloc proibido	Vagas fixas numeradas em vez de estacionamento livre.